

Python Tip: Product Iterators

Did you know that nested for loops make code slower? Try using iterators for cleaner, faster, and more efficient code.

The Problem with Nested Loops

Every time you make a for loop, it adds overhead to your computation.

```
import numpy as np

shape = (10, 10)
matrix = np.random.rand(*shape)

# Try to avoid nested loops
for row in range(matrix.shape[0]):
    for col in range(matrix.shape[1]):
        value = matrix[row, col]
        # Process value here

print(f"Processed {shape[0] * shape[1]} elements")
```

Using `itertools.product`

The `itertools.product()` function can replace nested loops. It generates all combinations of indices in a matrix.

```
from itertools import product
import numpy as np

shape = (10, 10)
matrix = np.random.rand(*shape)

# Replace nested loops with itertools.product()
for row, col in product(range(shape[0]), range(shape[1])):
    value = matrix[row, col]
    # Process value here

print(f"Processed {shape[0] * shape[1]} elements")
```

We replaced our double loop with a single loop!

Nested Loops of Unknown Depth

The `itertools.product()` function can also handle nested loops of unknown depth.

```
from itertools import product
import numpy as np

matrix_2d = np.random.rand(5, 5)
matrix_3d = np.random.rand(5, 5, 5)

# Loop over 2D matrix
count = 0
for indices in product(*map(range, matrix_2d.shape)):
    value = matrix_2d[indices]
    count += 1
print(f"2D matrix: {count} elements")

# Loop over 3D matrix
count = 0
for indices in product(*map(range, matrix_3d.shape)):
    value = matrix_3d[indices]
    count += 1
print(f"3D matrix: {count} elements")
```

```
2D matrix: 25 elements
3D matrix: 125 elements
```

Wrap-Up

Now you can replace nested loops with iterators for cleaner, more efficient code.

Follow me for more tips.

Shep Bryan IV